Pertemuan 7
Aljabar Linier Komputasional

Pemrograman untuk Ilmu Data dan Komputasi

MA25-21008 Kelas RA dan RB

Rifky Fauzi
Triyana Muliawati, S.Si., M.Si.

Program Studi Matematika, ITERA
Semester Ganjil 2026/2027

Fokus hari ini

- membaca vektor dan matriks sebagai struktur data;
- menulis algoritma manual sebelum memakai NumPy;
- menghitung banyak kerja dari loop;
- memakai residual untuk memeriksa hasil numerik.

Apa yang harus siap untuk LKM M7

Bagian	Yang dikerjakan	Ide algoritma
M7.1–M7.2	operasi vektor, norm, element-wise, $A@B$	loop komponen dan operasi array
M7.3–M7.4	perkalian matriks manual dan transpose	loop bersarang, cek dimensi
M7.5–M7.8	invers 2x2, SPL, residual	determinan, solve, verifikasi $Ax \approx b$
M7.9–M7.10	eigen dan data sebagai matriks	cek $Av \approx \lambda v$, mean kolom, Gram matrix

Pola kerja

Tuliskan rumus matematika, ubah menjadi indeks, baru tulis loop. Setelah itu, jika perlu, bandingkan dengan NumPy.

Vektor: operasi komponen dan norm

Contoh dari LKM

$$x = (1, -2, 3), \qquad y = (4, 0, -1)$$

$$x + y = (5, -2, 2), \qquad 2x - y = (-2, -4, 7)$$

$$x \cdot y = \sum_{i=1}^{n} x_i y_i, \qquad \|x\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$$

```python
def dot(x, y):
    total = 0
    for i in range(len(x)):
        total += x[i] * y[i]
    return total

# norm(x) = sqrt(dot(x,x))
```

Banyak kerja

Loop berjalan n kali.

$$T(n) = c_0 + c_1 n \quad \Rightarrow \quad O(n)$$

Element-wise, matrix product, dan operator @

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad B = \begin{pmatrix} 2 & 0 \\ 1 & 5 \end{pmatrix}$$

$$A \odot B = \begin{pmatrix} 1 \cdot 2 & 2 \cdot 0 \\ 3 \cdot 1 & 4 \cdot 5 \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 3 & 20 \end{pmatrix}$$

$$AB = \begin{pmatrix} 1 \cdot 2 + 2 \cdot 1 & 1 \cdot 0 + 2 \cdot 5 \\ 3 \cdot 2 + 4 \cdot 1 & 3 \cdot 0 + 4 \cdot 5 \end{pmatrix} = \begin{pmatrix} 4 & 10 \\ 10 & 2 \end{pmatrix}$$

```python
import numpy as np
A = np.array([[1,2],[3,4]])
B = np.array([[2,0],[1,5]])

A * B # element-wise
A @ B # perkalian matriks
```

Perbedaan penting

`*` bekerja per elemen dengan posisi yang sama.
`@` memakai baris-kali-kolom.

Perkalian matriks manual: tiga loop

Untuk $A \in \mathbb{R}^{m \times p}$ dan $B \in \mathbb{R}^{p \times n}$,

$$C_{ij} = \sum_{k=1}^{p} A_{ik} B_{kj}.$$

```python
def matmul_manual(A, B):
    m, p = len(A), len(A[0])
    p2, n = len(B), len(B[0])
    if p != p2:
        raise ValueError("dimensi tidak cocok")
    C = [[0 for j in range(n)] for i in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C
```

Menghitung T

Operasi terdalam berjalan untuk semua pasangan (i, j, k).

$$T(m, p, n) = m \cdot n \cdot p$$

Jika matriks persegi $n \times n$:

$$T(n) = n^3 \quad \Rightarrow \quad O(n^3).$$

Transpose: menukar peran baris dan kolom

Jika $A \in \mathbb{R}^{m \times n}$, maka $A^T \in \mathbb{R}^{n \times m}$ dan

$$(A^T)_{ij} = A_{ji}.$$

```
def transpose(A):
    m, n = len(A), len(A[0])
    T = [[0 for i in range(m)] for j in range
        (n)]
    for i in range(m):
        for j in range(n):
            T[j][i] = A[i][j]
    return T
```

Contoh

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

Banyak kerja

Setiap elemen dipindahkan sekali.

$$T(m, n) = mn \quad \Rightarrow \quad O(mn)$$

Invers 2x2 dan SPL

Invers matriks 2x2

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \qquad \det(A) = ad - bc$$

Jika $\det(A) \neq 0$,

$$A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}.$$

Jika $\det(A) = 0$, matriks singular dan invers tidak ada.

SPL sebagai $Ax = b$

$$A = \begin{pmatrix} 2 & 1 \\ 5 & 3 \end{pmatrix}, \qquad b = \begin{pmatrix} 1 \\ 4 \end{pmatrix}$$

$$\det(A) = 2 \cdot 3 - 1 \cdot 5 = 1 \neq 0.$$

Untuk 2x2, rumus invers manual masih masuk akal. Untuk ukuran besar, gunakan `np.linalg.solve(A,b)`, bukan membentuk invers terlebih dahulu.

Residual: cara memeriksa solusi numerik

Solusi numerik $\hat{x}$ tidak cukup hanya dilihat nilainya. Periksa sisa persamaan:

$$r = b - A\hat{x}, \qquad \|r\|_2.$$

```
def matvec(A, x):
    m, n = len(A), len(A[0])
    y = [0 for i in range(m)]
    for i in range(m):
        for j in range(n):
            y[i] += A[i][j] * x[j]
    return y
```

Banyak kerja

Matrix-vector product memakai dua loop.

$$T(m, n) = mn$$

Menghitung $r = b - A\hat{x}$ dan norm menambah orde $O(m)$.

$$T(m, n) = mn + m \quad \Rightarrow \quad O(mn)$$

Makna residual

Nilai eigen dan verifikasi

Untuk matriks kecil, nilai eigen dapat dihitung dari

$$\det(A - \lambda I) = 0.$$

Contoh:

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \quad A - \lambda I = \begin{pmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{pmatrix}$$

$$\det(A - \lambda I) = (2-\lambda)^2 - 1 = 0$$

$$\lambda = 1 \quad \text{atau} \quad \lambda = 3.$$

Yang dicek di komputer

Untuk setiap pasangan (λ, v) dari `np.linalg.eig`, hitung

$$\|Av - \lambda v\|_2.$$

Untuk $A \in \mathbb{R}^{n \times n}$, verifikasi Av membutuhkan $O(n^2)$.

Data sebagai matriks

Misalkan $X \in \mathbb{R}^{m \times d}$: m observasi dan d fitur.

$$\bar{x}_j = \frac{1}{m} \sum_{i=1}^{m} X_{ij}, \qquad (X_c)_{ij} = X_{ij} - \bar{x}_j.$$

$$G = X_c^T X_c.$$

Algoritma

1. hitung rata-rata tiap kolom;
2. kurangi setiap kolom dengan rata-ratanya;
3. hitung $X_c^T X_c$;
4. periksa G simetris.

Banyak kerja

Mean kolom dan centering:

$$O(md)$$

Gram matrix:

$$G_{jk} = \sum_{i=1}^{m} (X_c)_{ij} (X_c)_{ik}$$

Ringkasan algoritma dan orde

Masalah	Operasi utama	Orde
Dot product, norm vektor	loop semua komponen	$O(n)$
Transpose $m \times n$	pindahkan tiap elemen	$O(mn)$
Matrix-vector product	dua loop baris-kolom	$O(mn)$
Matrix product $m \times p$ dengan $p \times n$	tiga loop	$O(mpn)$
Matrix product persegi	tiga loop ukuran n	$O(n^3)$
Verifikasi eigen $Av \approx \lambda v$	matrix-vector product	$O(n^2)$
Mean dan centering data $m \times d$	semua entri data	$O(md)$
Gram matrix $X_c^T X_c$	fitur-pasangan-baris	$O(md^2)$

Kebiasaan yang dipakai di LKM

Jawaban numerik perlu disertai cara cek: dimensi cocok, hasil manual kecil, hasil NumPy, dan residual/norm selisih.

Arah kerja setelah kelas

Saat mengerjakan LKM M7

- Untuk soal kecil, hitung manual terlebih dahulu.
- Untuk list of lists, tulis loop dan cek dimensi.
- Untuk NumPy, bedakan `*`, `@`, dan `np.linalg.solve`.
- Untuk SPL dan eigen, selalu verifikasi dengan residual atau norm selisih.

Pertanyaan yang harus selalu dijawab

- Ukuran inputnya apa?
- Operasi terdalamnya apa?
- Berapa kali operasi itu dieksekusi?
- Apakah hasilnya bisa diperiksa kembali?

$$\text{rumus} \rightarrow \text{indeks} \rightarrow \text{loop} \rightarrow \text{cek hasil}$$